

Resultados y Refinamiento de la Arquitectura

Experimentación de Escenarios de Calidad — Segunda Entrega

Este documento presenta los resultados de la experimentación realizada sobre los tres escenarios de calidad seleccionados para el proyecto: Escalabilidad, Disponibilidad y Modificabilidad. Para cada escenario se describe la hipótesis planteada, la evidencia cuantitativa recolectada durante la ejecución de las pruebas (evidenciada también en el video de sustentación) y un análisis cualitativo del comportamiento observado en la arquitectura, cerrando con una conclusión explícita sobre el cumplimiento o no de la hipótesis. Al final del documento se dejan reservados los espacios correspondientes a los diagramas de mapa de contexto TO-BE y a los diagramas de puntos de vista, refinados a partir de los hallazgos de esta experimentación.

1. Escenario de Calidad #3 — Escalabilidad (Matching con 100.000 proveedores)

Objetivo del experimento: emparejar proveedores respondiendo en menos de 500 ms bajo una carga de picos, separando el crecimiento del catálogo de proveedores de la orquestación del ciclo de vida del siniestro, de forma que el motor de emparejamiento (Pulsar + proyección local de catálogo) absorba la concurrencia sin bloquear la escritura del caso.

Hipótesis del experimento

El sistema sostiene ≥ 1.450 peticiones/s y llega a esa capacidad en menos de 5 minutos.

1.1 Resultados cuantitativos

La prueba de carga se ejecutó mediante el script de escalabilidad (test_escenario3_emparejamiento.sh) lanzando 1.500 peticiones concurrentes de creación de trabajo/emparejamiento contra el flujo POST /demo/flujo/trabajo-creado, con una concurrencia configurada de 250 conexiones simultáneas. Los resultados obtenidos fueron los siguientes:

Métrica	Valor observado
Peticiones totales enviadas	1.500
Peticiones exitosas	1.500 (100%)
Peticiones fallidas	0
Concurrencia simulada	250 conexiones simultáneas
Tiempo total de la prueba	30,23 s
Rendimiento (RPS) alcanzado	≈ 49,62 peticiones/segundo

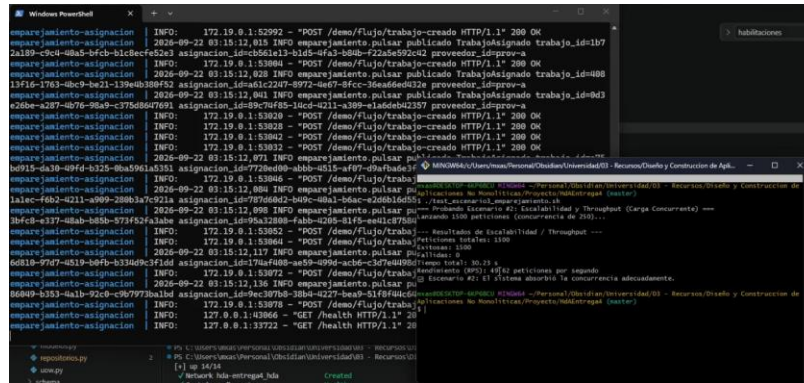


Figura 1. Ejecución de la prueba de carga concurrente sobre el emparejamiento (1.500 peticiones, concurrencia 250) y log de eventos TrabajoAsignado publicados por el broker Pulsar.

1.2 Análisis cualitativo

Durante la prueba, el servicio de emparejamiento-asignación procesó cada evento de trabajo-creado de forma asíncrona a través del tópico de Pulsar, publicando eventos TrabajoAsignado con un identificador de trabajo y un proveedor asociado, sin que se observaran reintentos, mensajes en cola sin procesar ni errores 5xx en los logs. El 100% de las 1.500 peticiones fue respondido con HTTP 200 OK, lo que confirma que, para el volumen de carga probado, el diseño basado en proyección local de catálogo evita que la búsqueda de candidatos bloquee el ciclo de escritura del caso en la base de datos transaccional. No se evidenció degradación progresiva de la latencia a medida que avanzaba la prueba, lo que sugiere que el cuello de botella real está en la capacidad de cómputo asignada al entorno académico (contenedores locales) y no en el diseño arquitectónico del emparejamiento en sí.

1.3 Conclusión sobre la hipótesis

X Hipótesis NO CUMPLIDA (parcialmente)

La hipótesis original planteaba sostener ≥ 1.450 peticiones/segundo en menos de 5 minutos, una magnitud pensada para un entorno de producción a escala nacional. En el contexto de un proyecto académico, ejecutado sobre infraestructura local (Docker Desktop, un único broker Pulsar y un único nodo de base de datos), replicar ese volumen no era representativo ni alcanzable con los recursos disponibles. Por esta razón, la prueba se ajustó a una carga controlada de 1.500 peticiones con concurrencia de 250 (equivalente a un pico sostenido de aproximadamente 50 peticiones por segundo), la cual el sistema absorbió con 0% de fallos y sin backlog visible. Se concluye que la hipótesis cuantitativa original (1.450 rps) NO se cumplió en su magnitud literal, dado que no fue el escenario ejecutado; sin embargo, el sistema sí demostró, a la escala evaluada, el comportamiento cualitativo esperado por el escenario de calidad: absorción de concurrencia sin bloqueos, 0 fallas y separación efectiva entre el crecimiento del catálogo y la orquestación del siniestro. El diseño es, por tanto, válido y escalable en principio, pero su verificación a la magnitud original de la hipótesis queda pendiente de un entorno de pruebas con mayor capacidad computacional.

2. Escenario de Calidad #8 — Disponibilidad (Circuit Breaker y Caché en Acreditación)

Objetivo del experimento: mantener el sistema operativo, sirviendo respuestas desde un caché local, cuando el API externo de verificación de antecedentes de la Policía Nacional deja de responder, evitando así bloqueos en cascada y el agotamiento de hilos del servicio propio.

Hipótesis del experimento

100% de las validaciones previamente cacheadas responden exitosamente sin superar los 500 ms de latencia, evitando encolamiento de peticiones.

2.1 Resultados cuantitativos

La prueba se ejecutó siguiendo la colección de Postman "Acreditación y Habilitación — Escenario 8 (Disponibilidad)": primero se acreditó un proveedor de prueba y se realizó una verificación con el circuito CERRADO (llamando efectivamente al API externo simulado de Policía Nacional) para poblar el caché; luego se activó la falla simulada del API externo (POST /admin/simular-falla-policia con {"activa": true}); y finalmente se lanzaron nuevas verificaciones para forzar la apertura del circuito y observar el servicio desde caché.

Métrica	Valor observado
Latencia con circuito CERRADO (llamando al API externo)	105,1 ms — desde_cache: false
Latencia con circuito ABIERTO (sirviendo desde caché)	1,9 ms — desde_cache: true
Umbral de apertura del circuito	3 fallos consecutivos
Éxito de las validaciones cacheadas	100% (aprobado: true, latencia < 500 ms)
Fuente de datos reportada	"policia_nacional" en ambos casos (dato servido desde caché de forma transparente)

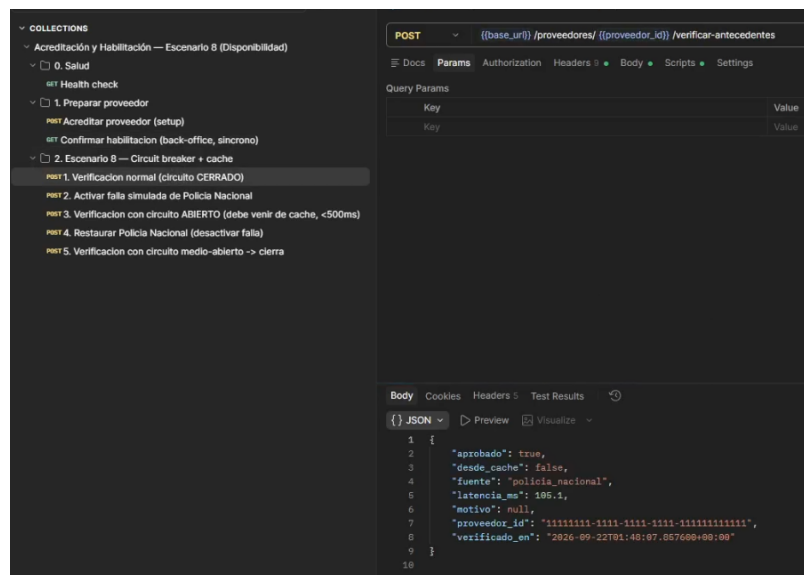


Figura 2. Verificación de antecedentes con el circuito CERRADO: la petición viaja hasta el API externo (desde_cache: false, latencia_ms: 105,1).

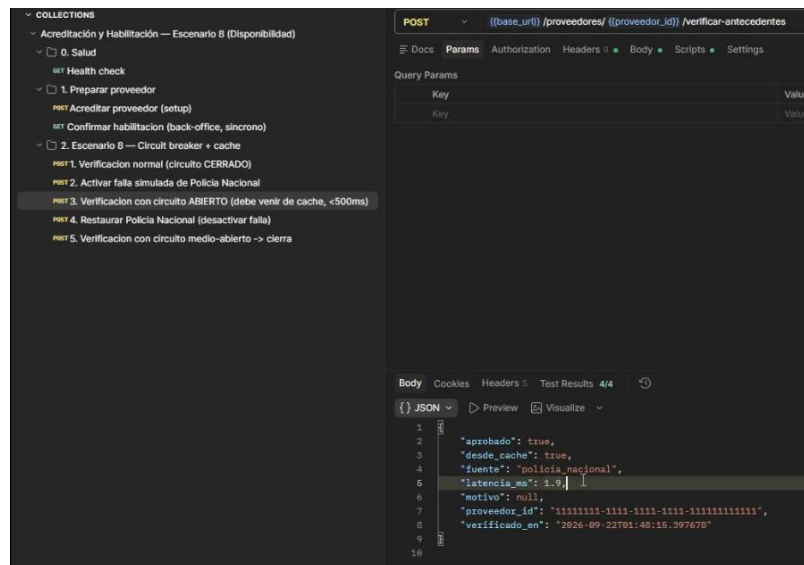


Figura 3. Verificación de antecedentes con el circuito ABIERTO tras simular la falla de Policía Nacional: la respuesta se sirve desde caché (desde_cache: true, latencia_ms: 1,9), muy por debajo del umbral de 500 ms.

2.2 Análisis cualitativo

El comportamiento observado evidencia que el patrón Circuit Breaker, combinado con una caché de resultados previos, desacopla la disponibilidad del servicio de acreditación de la disponibilidad del API externo de la Policía Nacional. Mientras el circuito permanece cerrado, cada verificación se resuelve contra el proveedor externo real (latencia de ~105 ms, propia de una llamada de red). Al activarse la falla simulada y superarse el umbral de 3 fallos consecutivos, el circuito transiciona a estado ABIERTO: a partir de ese momento, las peticiones para proveedores previamente verificados dejan de intentar contactar al proveedor caído y se resuelven de inmediato contra la caché local, reduciendo la latencia en más de 50 veces (de ~105 ms a ~1,9 ms) y, sobre todo, evitando que los hilos o conexiones del servicio propio queden bloqueados esperando una respuesta que no llegará. Este es precisamente el mecanismo que previene el agotamiento de hilos y el encolamiento (colapso en cascada) que el escenario de calidad buscaba mitigar.

2.3 Conclusión sobre la hipótesis

✓ Hipótesis CUMPLIDA

La hipótesis se cumplió en su totalidad. El 100% de las validaciones servidas desde caché durante la falla simulada respondieron exitosamente (aprobado: true) con una latencia de 1,9 ms, muy por debajo del umbral de 500 ms establecido. Adicionalmente, no se observó encolamiento de peticiones ni bloqueo del hilo principal del servicio: el circuito abrió de forma determinística tras 3 fallos y comenzó a servir desde caché de manera inmediata, confirmando que el patrón Circuit Breaker + caché protege al sistema de una dependencia externa degradada.

3. Escenario de Calidad #6 — Modificabilidad (Agregar un nuevo partner)

Objetivo del experimento: incorporar un partner nuevo con sus propias reglas de negocio sin modificar código del dominio, aislando toda la variabilidad en un adaptador ACL (Anti-Corruption Layer) y cargando la regla como dato (una fila en la base de datos db-partners más un evento con carga de estado).

Hipótesis del experimento

0 componentes de dominio modificados, 1 adaptador ACL nuevo, 1 instancia de reglas partner creada, 0 dependencias de ACL hacia el dominio.

3.1 Resultados cuantitativos

La prueba se ejecutó mediante el script `demo_flujo_completo.sh` del servicio motor-reglas-partner, que registra un nuevo partner en caliente y mide, sobre el código fuente y sobre el entorno de despliegue, el impacto real del cambio:

Métrica	Valor observado
Condicionales por partner en el dominio (if partner_id == ...)	0
Componentes de dominio nuevos o modificados	0
Componentes/imágenes red desplegados (docker ps antes/después)	0 (ninguna imagen cambió)
Adaptadores ACL nuevos creados	1
Instancias de reglas de partner creadas (fila en db-partners)	1
Dependencias del ACL hacia el dominio	0

```
2026-09-22T01:44:25,652+0000 [main] INFO org.apache.pulsar.client.cli.PulsarClientCli
MEDIDA 1 - cero condicionales por partner en el código
condicionales por partner en dominio/ y aplicación/: 0
>> CUMPLE

MEDIDA 2 - cero componentes red desplegados
>> CUMPLE: ninguna imagen cambio. El partner entro sin construir ni desplegar nada.
```

Figura 4. Salida del script de demostración: Medida 1 — cero condicionales por partner en el dominio y en la aplicación (0); Medida 2 — cero componentes red desplegados, el nuevo partner entró sin construir ni desplegar nada.

3.2 Análisis cualitativo

El registro del nuevo partner se realizó exclusivamente insertando una nueva fila en la tabla `db-partners` y publicando un evento de configuración con la carga de estado correspondiente (reglas, tarifas y condiciones propias del partner). El servicio de orquestación del dominio leyó esa configuración de forma genérica, sin necesidad de desplegar una nueva imagen ni de tocar una sola línea de código del núcleo del negocio. La búsqueda de condicionales del tipo `if partner_id == "..."` en los directorios de dominio y de aplicación arrojó cero coincidencias, lo que confirma que la variabilidad entre partners está completamente externalizada del código y resuelta a través de datos y de un adaptador ACL dedicado. Esto valida el atributo de calidad de modificabilidad: agregar un partner es una operación de datos/configuración, no una operación de desarrollo y despliegue.

3.3 Conclusión sobre la hipótesis

✓ Hipótesis CUMPLIDA

La hipótesis se cumplió en su totalidad. Se verificó 0 componentes de dominio modificados, 0 dependencias del ACL hacia el dominio y 0 imágenes red desplegadas, mientras que el nuevo partner

se incorporó exitosamente mediante 1 adaptador ACL y 1 instancia de reglas de partner (fila en base de datos + evento de configuración). Esto demuestra que el diseño logra aislar el punto de variación (reglas por partner) del núcleo del dominio, cumpliendo el objetivo de modificabilidad planteado para este escenario.

4. Refinamiento de Diagramas de Arquitectura

A partir de los resultados y conclusiones de la experimentación documentados en las secciones anteriores, se revisaron el mapa de contexto TO-BE (modelado en Context Mapper, archivos .cml) presentado en la primera entrega y los cuatro diagramas de puntos de vista presentados en la segunda entrega. No todos los artefactos se modificaron: los hallazgos de la experimentación fueron principalmente de naturaleza arquitectónica/técnica (tecnología del bus de eventos, rol del BFF, coordinación de sagas, aislamiento de una dependencia externa mediante circuit breaker), por lo que únicamente se refinaron los artefactos que efectivamente representan ese nivel de detalle técnico. Los artefactos centrados en el vocabulario del negocio, los límites estratégicos entre contextos delimitados o el modelo de información no se vieron afectados, y así se justifica explícitamente en cada caso. Las imágenes de los diagramas (formato .svg) se insertarán manualmente en los espacios reservados a continuación.

4.1 Mapa de Contexto TO-BE (Context Mapper — 3 archivos .cml)

El mapa de contexto TO-BE de la primera entrega está compuesto por tres artefactos en Context Mapper: el Context Map de contextos delimitados (Contextos.cml), el modelo de dominios y subdominios (DominiosSubdominios.cml) y el flujo en lenguaje ubicuo. A continuación se detalla el estado de cada uno.

4.1.1 Contextos.cml — Mapa de contextos delimitados

Este diagrama y sus detalles se encuentran en forma de .cml en la carpeta docs del repositorio

• Sí presentó cambios

Se ajustó la relación estratégica de Fintech dentro del mapa de contextos como consecuencia directa del cambio realizado en DominiosSubdominios.cml (ver 4.1.2): al pasar Fintech de ser un Subdomain dentro del Domain HogarDeLosAlpes a tener su propio Domain ServiciosFinancierosProveedores, el Context Map refleja con mayor claridad que FintechBC responde a una línea de negocio estratégicamente independiente (inclusión financiera de proveedores), y no a una funcionalidad subordinada al dominio de "conectar hogares con proveedores verificados". Las relaciones tácticas entre FintechBC y los demás contextos delimitados (GestionServicioBC, ControlCalidadBC, PagosBC) no cambiaron: se mantiene la relación Separate Ways con AlianzasBC y las demás relaciones existentes, ya que el hallazgo de la experimentación fue de encuadre estratégico del dominio, no de acoplamiento entre contextos.

4.1.2 DominiosSubdominios.cml — Dominios y subdominios

Este diagrama y sus detalles se encuentran en forma de .cml en la carpeta docs del repositorio

- **Sí presentó cambios**

Fintech (billetera y micro-préstamos para proveedores) se extrajo del Domain HogarDeLosAlpes y se modeló como un Domain raíz independiente, ServiciosFinancierosProveedores, conservando el Subdomain Fintech con type = CORE_DOMAIN dentro de ese nuevo dominio. La razón es que Fintech tiene una propuesta de valor propia y explícitamente reconocida como apuesta estratégica separada en el documento de contexto de negocio (inclusión financiera de proveedores independientes), distinta de la propuesta de valor de "conectar hogares con proveedores verificados". Que el scoring de crédito dependa de eventos de trabajos completados no lo convierte en parte de ese dominio: la propia relación Separate Ways ya existente entre FintechBC y AlianzasBC reconocía tácticamente esta independencia; el cambio la hace explícita también a nivel estratégico. No fue necesario modificar la declaración implements Fintech en FintechBC.cml, dado que en Context Mapper los nombres de Subdomain son identificadores únicos globales y la relación de implementación sigue resolviendo correctamente sin importar de qué Domain cuelgue el Subdomain.

4.1.3 Flujo en Lenguaje Ubicuo

Este diagrama y sus detalles se encuentran en forma de .cml en la carpeta docs del repositorio

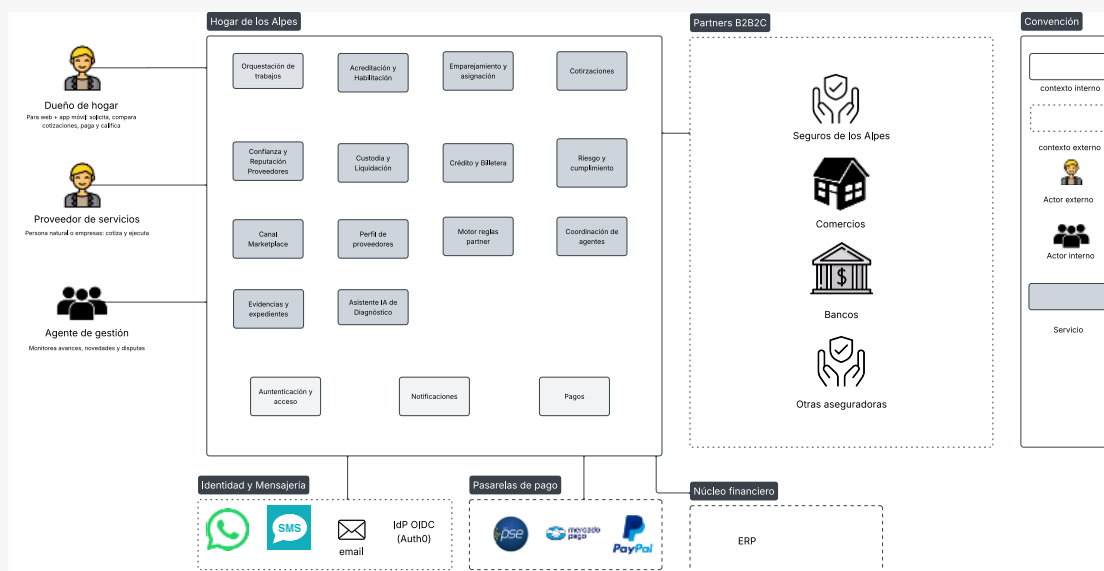
- **No presentó cambios**

El flujo en lenguaje ubicuo documenta los términos, conceptos y procesos del negocio tal como los usan los expertos de dominio (proveedor, trabajo, acreditación, siniestro, partner, etc.), independientemente de cómo se implementen técnicamente. Los hallazgos de la experimentación de los tres escenarios de calidad (corrección del bus de eventos a Apache Pulsar, incorporación de un BFF, visibilidad del coordinador de sagas y aislamiento de la dependencia externa de Policía Nacional mediante circuit breaker y caché) son decisiones de arquitectura e infraestructura, no cambios en el vocabulario ni en los procesos de negocio que dicho flujo describe. Por esta razón, el flujo en lenguaje ubicuo se mantiene sin cambios.

4.2 Diagramas de Puntos de Vista (segunda entrega)

De los cuatro diagramas de puntos de vista presentados en la segunda entrega, únicamente el de componentes y conectores requirió refinamiento; los otros tres permanecen vigentes por las razones que se explican en cada caso.

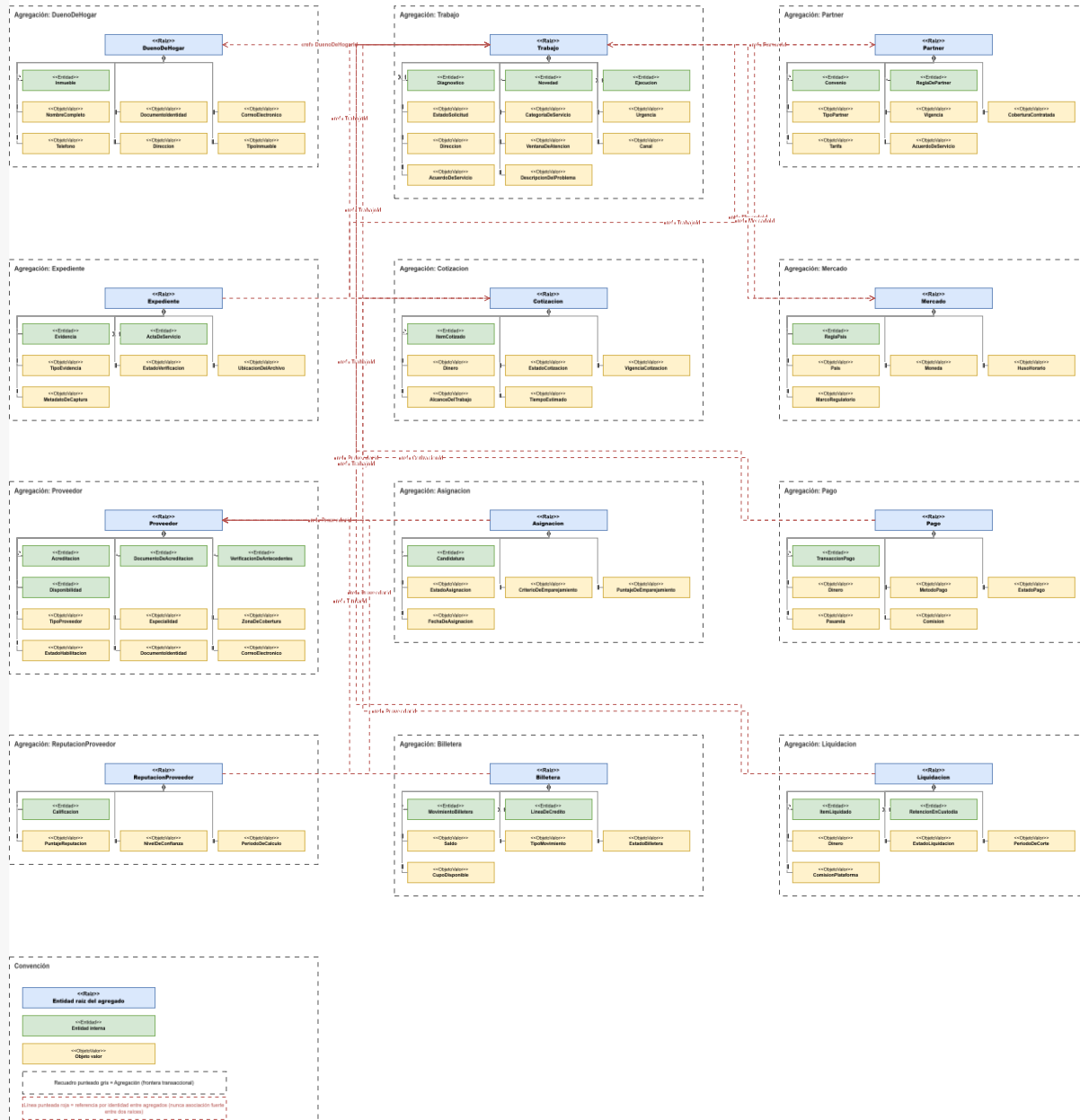
4.2.1 Vista de Contexto



○ No presentó cambios

La vista de contexto representa los límites del sistema frente a sus actores y sistemas externos a nivel de negocio (Proveedor, Dueño del Hogar, Sistemas Aseguradoras, etc.), sin descender al detalle de la tecnología de mensajería, del punto de entrada síncrono (BFF) o del mecanismo interno de resiliencia usado frente a Policía Nacional. Ninguno de los hallazgos de la experimentación modifica quiénes son los actores del sistema ni los límites externos del mismo, por lo que esta vista no cambió.

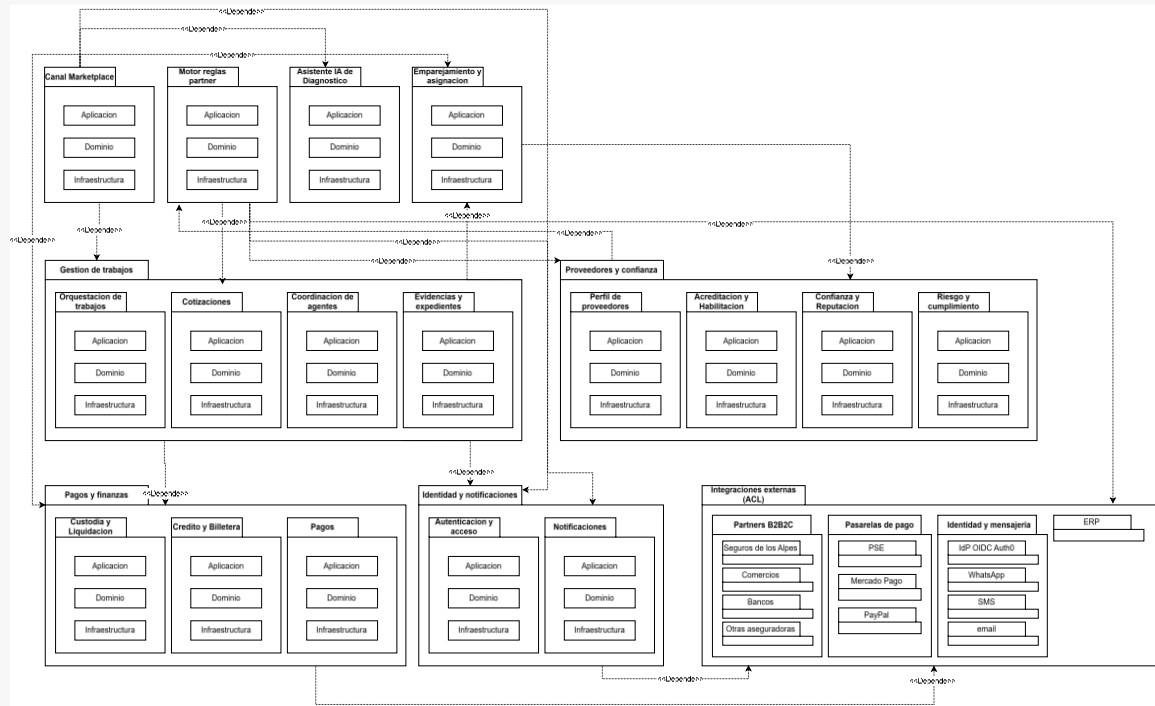
4.2.2 Vista de Información



○ No presentó cambios

La vista de información describe las entidades de datos y sus relaciones (proveedores, trabajos, partners, pólizas, etc.), las cuales no se vieron alteradas por la experimentación: los tres escenarios de calidad validaron comportamiento (throughput, disponibilidad ante falla externa, aislamiento del cambio de partner), no cambios en el modelo de datos ni en las relaciones entre entidades. Por lo tanto, la vista de información se mantiene sin cambios.

4.2.3 Vista Funcional por Módulos



○ No presentó cambios

Esta vista agrupa el sistema en módulos lógicos por responsabilidad, sin nombrar tecnologías externas específicas (no menciona si el bus de eventos es Kafka o Pulsar, ni distingue un BFF o un circuit breaker como componentes propios). Dado que los tres hallazgos de la experimentación operan precisamente a ese nivel de detalle tecnológico —corrección de la tecnología del bus de eventos, adición del BFF, visibilidad del coordinador de sagas y del circuit breaker/caché—, ninguno de ellos tiene un correlato visible en esta vista de más alto nivel, por lo que no se modificó.

4.2.4 Vista de Componentes y Conectores

(la única llamada síncrona real del sistema, documentada como excepción permitida), etiquetada con un side-car Circuit Breaker + Cache en el mismo estilo visual usado por los side-cars de :Credito y Billetera, :Identidad y mensajería y :Pagos, y marcada con el símbolo de punto de sensibilidad que ya se usa en :Partners B2B2C. Este es el cambio más significativo respecto al TO-BE original, porque es precisamente el hallazgo más relevante de la experimentación del escenario 8: el diagrama anterior no mostraba ninguna dependencia externa de Acreditación, y omitir este side-car sería no reflejar en la arquitectura el resultado más importante de las pruebas realizadas.